Task Mini Project 7

Oct 31, 2026 · @Bayuzen Ahmad

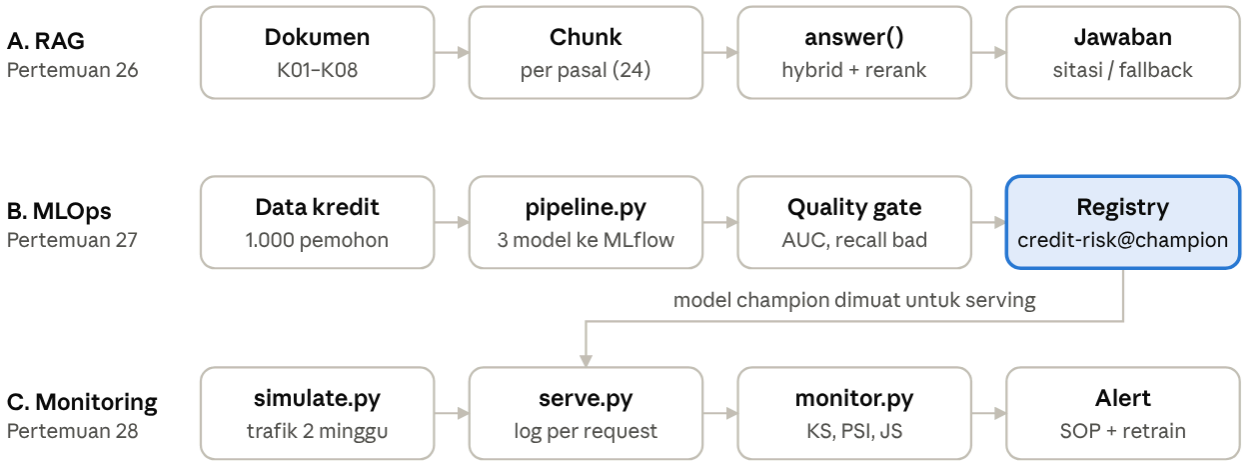
Ringkasan

Kamu membangun satu repo kecil untuk bank fiktif bernama Bank Nusantara. Isinya tiga bagian: asisten tanya-jawab kebijakan kredit (RAG), model machine learning untuk menilai risiko kredit beserta alur MLOps-nya, dan monitoring model setelah dipakai. Template hanya berisi kerangka arsitektur: struktur folder, nama file, nama fungsi, dan penjelasan input/output setiap fungsi. Kamu menulis seluruh implementasinya, menjalankannya, lalu menjelaskan hasilnya.

Bagian	Materi	Yang kamu kerjakan	Data
A. RAG	Pertemuan 26	Chunking per pasal, pencarian dense + BM25 + hybrid + rerank, sitasi, fallback, evaluasi	8 dokumen kebijakan kredit (sudah ada di template)
B. MLOps	Pertemuan 27	Melatih 3 model credit risk, mencatat di MLflow, versi data, quality gate, registry, promote model terbaik	German Credit Data with Risk
C. Monitoring	Pertemuan 28	Serving yang mencatat log, simulasi trafik 2 minggu, drift KS/PSI/Jensen-Shannon, alert, dashboard, SOP	Data yang sama, ditambah simulasi drift

- **Bentuk:** individu
- **Deadline:** 1 minggu sejak tugas dibagikan
- **Kebutuhan:** laptop biasa, cukup CPU. Tidak perlu GPU, API key, Docker, atau GitHub Actions.
- **Yang dinilai:** kode yang berjalan, angka yang jujur, dan penjelasan keputusanmu. Skor tertinggi bukan syarat lulus.

Model champion dari Bagian B adalah model yang dipantau di Bagian C



alur tugas · 3 bagian

Bagian A berdiri sendiri. Bagian B menghasilkan model `credit-risk@champion`, dan model itulah yang dilayani serta dipantau di Bagian C.

Persiapan

1. **Ambil template** dari mentor (`template.zip`), ekstrak, lalu jadikan repository GitHub bernama `tugas-26-28-<nama>`.
2. **Unduh dataset** German Credit Data with Risk dari Kaggle. Simpan `german_credit_data.csv` di `data/credit/`. Isinya 1.000 pemohon kredit, 9 fitur, dan label `Risk` (good/bad).
3. **Buat environment** dengan Python 3.10–3.13:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # Mac/Linux
pip install -r requirements.txt
```

Saat pertama kali dijalankan, Bagian A mengunduh dua model dari Hugging Face (sekitar 1 GB, sekali saja): `paraphrase-multilingual-MiniLM-L12-v2` untuk embedding dan `mmarco-mMiniLMv2-L12-H384-v1` untuk rerank. Keduanya mengerti Bahasa Indonesia dan jalan di CPU.

Isi template hanya **kerangka arsitektur**: struktur folder, nama file, nama fungsi, dan docstring yang menjelaskan input, output, serta langkah yang harus dilakukan. Setiap fungsi masih berisi `raise NotImplementedError`, dan tugasmu adalah menulis isinya.

Jangan mengubah nama fungsi atau bentuk output-nya, karena keduanya dipakai oleh file lain dan oleh test.

File	Fungsi yang kamu tulis	Bagian
`src/rag.py`	`load_chunks`, `tokenize`, `_index`, `dense_search`, `bm25_search`, `rrf`, `rerank`, `answer`	A
`src/rag_eval.py`	`hit_at_k`, `summarize`, `main`	A
`src/data.py`	`load_data`, `split`, `data_version`, `validate_record`	B dan C
`src/train.py`	`build_model`, `evaluate`	B
`src/quality_gate.py`	`check`, `main`	B
`src/pipeline.py`	`setup_mlflow`, `step_test`, `step_train`, `step_gate`, `step_select`, `step_register_promote`, `main`	B
`src/serve.py`	`decide`, `CreditScorer.__init__`, `CreditScorer.predict`	C
`src/simulate.py`	`make_week2`, `main`	C
`src/monitor.py`	`psi`, `js_distance`, `drift_table`, `ops_metrics`, `alerts`, `dashboard`, `main`	C

Folder `tests/` sudah berisi 8 test yang menjadi patokan kebenaran. Semuanya harus lulus.

Urutan menjalankan setelah semua fungsi selesai:

```
python -m pytest -q                              # 8 test harus lulus
python src/rag_eval.py                           # Bagian A
python src/pipeline.py --reason "commit baru"  # Bagian B
python src/quality_gate.py                       # Bagian B: exit code 0 / 1
python src/simulate.py                           # Bagian C
python src/monitor.py                            # Bagian C
mlflow ui --backend-store-uri sqlite:///mlflow.db   # buka
http://127.0.0.1:5000
```

Istilah Bagian A (RAG)

Baca bagian ini sebelum mengerjakan Bagian A. Setiap istilah diberi contoh kecil dari dokumen kebijakan kredit.

RAG (Retrieval-Augmented Generation). Sistem yang lebih dulu mencari potongan dokumen yang relevan, baru menjawab berdasarkan potongan itu. Ibaratnya ujian open book: sebelum menjawab, kamu membuka buku ke halaman yang tepat.

Chunk dan chunking. Chunk adalah potongan kecil dokumen, dan chunking adalah proses memotongnya. Di tugas ini satu pasal menjadi satu chunk, supaya setiap potongan berisi satu aturan yang utuh (document-aware). Karena ada 8 dokumen dengan 3 pasal masing-masing, hasilnya 24 chunk. Contoh untuk K06:

chunk_id	doc_id	pasal	text
K06-P1	K06	1	Keterlambatan pembayaran cicilan dikenakan denda 0,1% per hari…
K06-P2	K06	2	Kredit yang menunggak lebih dari 90 hari digolongkan sebagai kredit macet.
K06-P3	K06	3	Nasabah yang mengalami kesulitan bayar dapat mengajukan restrukturisasi…

Embedding. Teks yang diubah menjadi deretan angka (vektor) yang mewakili maknanya. Dua kalimat yang maknanya mirip menghasilkan vektor yang berdekatan, walaupun kata-katanya berbeda.

Cosine similarity. Angka yang mengukur seberapa mirip dua vektor, dari −1 sampai 1. Makin dekat ke 1, makin mirip maknanya. Contoh untuk pertanyaan "Berapa denda kalau telat bayar?" (angkanya ilustrasi):

Chunk	Isi singkat	Cosine similarity
K06-P1	denda keterlambatan 0,1% per hari	0,61
K08-P2	pengaduan dijawab 10 hari kerja	0,15

Dense retrieval. Mencari chunk berdasarkan kemiripan makna, memakai embedding dan cosine similarity. Cara ini kuat untuk sinonim: "telat bayar" tetap menemukan pasal tentang "keterlambatan pembayaran".

Sparse retrieval (BM25). Mencari chunk berdasarkan kata yang sama persis. Kata yang jarang muncul diberi bobot lebih besar. Cara ini kuat untuk istilah khusus dan angka seperti "NPWP" atau "90 hari", tetapi lemah untuk sinonim.

Hybrid dan RRF. Hybrid menggabungkan hasil dense dan BM25. RRF (Reciprocal Rank Fusion) menggabungkannya memakai peringkat, bukan skor, karena skala skor kedua metode berbeda. Rumusnya, dengan k = 60 dan rank dimulai dari 1:

```
skor_RRF(d) = 1 / (60 + rank_dense(d))  +  1 / (60 + rank_BM25(d))
```

Contoh dengan 3 chunk:

Chunk	Rank dense	Rank BM25	Skor RRF
A	1	5	1/61 + 1/65 = 0,0318
B	2	1	1/62 + 1/61 = 0,0325
C	3	2	1/63 + 1/62 = 0,0320

Urutan akhirnya B, C, A. Chunk A juara di dense tetapi jauh di BM25, sedangkan B bagus di keduanya. Contoh ini juga dipakai di `tests/test_rag.py`.

Reranking (cross-encoder). Tahap penyaringan kedua. Dense dan BM25 menilai pertanyaan dan chunk secara terpisah, sehingga cepat tetapi kasar. Cross-encoder membaca pertanyaan dan chunk sekaligus, sehingga lebih teliti tetapi lebih lambat. Karena itu reranker hanya dipakai untuk 8 kandidat teratas hasil hybrid, lalu mengurutkannya ulang.

top_k dan top_score. top_k adalah jumlah chunk teratas yang diambil; di tugas ini 3. top_score adalah cosine similarity chunk peringkat 1 dari dense retrieval, dan angka ini dipakai untuk fallback.

Sitasi. Sumber jawaban yang ditulis di akhir jawaban, misalnya `[Sumber: K06, Pasal 1]`. Dengan sitasi, pengguna bisa mengecek jawaban ke kebijakan aslinya.

Fallback dan ambang batas (threshold). Fallback adalah jawaban cadangan saat sistem tidak yakin. Kalau top_score di bawah ambang batas, sistem menjawab "informasi tidak ada di dokumen", bukan mengarang. Contoh dengan ambang 0,35:

Pertanyaan	top_score	Hasil
Berapa denda kalau telat bayar?	0,61	Dijawab dengan sitasi
Bagaimana cara membuat nasi goreng?	0,11	Fallback

Hit@3. Persentase pertanyaan yang dokumen benarnya muncul di 3 hasil teratas. Dokumennya tidak harus peringkat 1; cukup masuk 3 besar. Contoh dengan 4 pertanyaan:

Pertanyaan	Dokumen benar	3 hasil teratas	Masuk 3 besar?
Q1	K01	K01, K03, K02	Ya

Pertanyaan	Dokumen benar	3 hasil teratas	Masuk 3 besar?
Q2	K05	K06, K05, K01	Ya
Q3	K07	K02, K04, K08	Tidak
Q4	K04	K03, K01, K04	Ya

Hit@3 = 3 / 4 = 0,75.

Citation accuracy. Persentase pertanyaan dalam topik yang sumber peringkat 1-nya tepat dan tidak terkena fallback. Pada contoh di atas hanya Q1 yang dokumen benarnya di peringkat 1, jadi citation accuracy = 1 / 4 = 0,25, walaupun Hit@3-nya 0,75. Sitasi selalu diambil dari peringkat 1, jadi angka inilah yang dilihat pengguna.

Fallback accuracy. Persentase pertanyaan luar topik yang benar-benar dijawab dengan fallback. Kalau 2 dari 3 pertanyaan luar topik ditolak, nilainya 2 / 3 = 0,67.

Latency. Waktu dari pertanyaan masuk sampai jawaban keluar, dalam milidetik (ms). 1.000 ms sama dengan 1 detik.

Bagian A — RAG kebijakan kredit (Pertemuan 26)

Targetnya: mode `hybrid_rerank` mencapai **Hit@3 ≥ 0,90** pada 12 pertanyaan dalam topik, dan **menolak ketiga pertanyaan luar topik** (fallback accuracy 1,0). Analis kredit Bank Nusantara memakai asisten ini untuk bertanya soal syarat, plafon, bunga, denda, dan keputusan kredit.

Data sudah ada di template: `data/rag/docs/K01.md` sampai `K08.md`, dan `data/rag/eval.csv` (15 pertanyaan, 3 di antaranya bertanda `NONE` karena di luar topik).

1. **Chunking** (`rag.py` → `load_chunks`). Baca setiap file `.md`, kenali baris `Pasal n: teks` dengan regex, lalu simpan `chunk_id`, `doc_id`, `pasal`, `title`, dan `text`. Hasilnya harus 24 chunk.
2. **Index** (`tokenize`, `_index`). Siapkan semua komponen pencarian sekali saja dengan `functools.lru_cache`: embedding semua chunk (dinormalisasi), BM25 dari teks yang sudah di-tokenize, dan model reranker.
3. **Dense dan BM25** (`dense_search`, `bm25_search`). Pada dense, cosine similarity cukup dihitung dengan dot product karena vektornya sudah dinormalisasi. Keduanya mengembalikan index chunk yang terurut dari paling relevan.
4. **Hybrid** (`rrf`). Terapkan rumus RRF. Test `test_rrf_contoh_modul` memakai contoh tabel di atas.
5. **Rerank** (`rerank`). Beri skor setiap pasangan (pertanyaan, chunk) dengan `reranker.predict`, lalu urutkan kandidat dari skor terbesar.

6. **Jawaban, sitasi, dan fallback** (`answer`). Ikuti aturan di docstring: pertanyaan kosong langsung fallback tanpa memuat model, `top_score` di bawah ambang juga fallback, dan selain itu jawab dengan teks chunk peringkat 1 ditambah `[Sumber: Kxx, Pasal n]` .
7. **Evaluasi** (`rag_eval.py` → `hit_at_k` , `summarize` , `main`). Jalankan `python src/rag_eval.py` . Hasilnya `reports/rag_eval_summary.csv` untuk keempat mode: `dense` , `bm25` , `hybrid` , `hybrid_rerank` .
8. **Tentukan ambang fallback.** Buka `reports/rag_eval_detail.csv` , lalu bandingkan `top_score` terendah dari pertanyaan dalam topik dengan `top_score` tertinggi dari pertanyaan luar topik. Ambang yang aman ada di antara keduanya. Nilai awal di template adalah 0,35. Tulis angka akhirmu beserta alasannya di README.

Perbandingan keempat mode adalah controlled experiment: dokumen, pertanyaan, top_k, dan ambang dibuat sama, dan hanya mode pencarian yang berbeda. Di README, jelaskan mode mana yang kamu pilih dengan mempertimbangkan akurasi **dan** latency. Rerank biasanya paling akurat tetapi juga paling lambat.

Istilah Bagian B (machine learning dan MLOps)

Machine learning untuk credit risk

Credit risk dan label. Credit risk adalah kemungkinan peminjam gagal membayar. Di dataset German Credit, setiap pemohon diberi label `Risk` : `good` (lancar) atau `bad` (bermasalah). Template mengubahnya menjadi kolom `bad` : 1 untuk bad dan 0 untuk good. Dari 1.000 pemohon, 300 berlabel bad.

Fitur numerik dan kategori. Fitur adalah kolom yang dipakai model untuk menebak label. Fitur numerik berupa angka (`age` , `credit_amount` , `duration`). Fitur kategori berupa pilihan (`sex` , `job` , `housing` , `saving_accounts` , `checking_account` , `purpose`). Nilai kosong (NA) di `saving_accounts` dan `checking_account` diisi `unknown` , karena kosong di sini berarti tidak punya atau tidak diketahui.

Train/test split (stratified). Data dibagi 70% untuk melatih model dan 30% untuk mengujinya. Stratified berarti proporsi bad di kedua bagian sama-sama 30%, supaya pengujiannya adil. Data training juga dipakai sebagai **referensi** di Bagian C.

Preprocessing pipeline. Langkah penyiapan data yang menempel pada model. StandardScaler menyamakan skala angka, dan OneHotEncoder mengubah kategori menjadi kolom 0/1. Karena keduanya disimpan bersama model, data saat serving diproses dengan cara yang persis sama.

Probabilitas dan threshold. Model mengeluarkan `prob_bad`, yaitu peluang pemohon bermasalah, dari 0 sampai 1. Untuk metrik di Bagian B, prob_bad ⩾ 0,5 dianggap prediksi bad. Untuk keputusan kredit di Bagian C, dipakai kebijakan K05: di bawah 0,3 disetujui, 0,3 sampai 0,6 ditinjau analis, dan di atas 0,6 ditolak.

Recall, precision, dan F1 (kelas bad). Contoh 10 pemohon, 3 di antaranya benar-benar bad:

	Model bilang bad	Model bilang good
Benar-benar bad (3)	2 (tertangkap)	1 (lolos)
Benar-benar good (7)	2 (salah tuduh)	5

- **Recall bad** = 2 / 3 = 0,67. Dari semua pemohon bermasalah, berapa yang tertangkap. Bagi bank, pemohon bad yang lolos berarti uang hilang.
- **Precision bad** = 2 / 4 = 0,50. Dari semua yang dituduh bad, berapa yang memang bad. Precision rendah berarti banyak nasabah baik yang ditolak.
- **F1** = 2 × 0,50 × 0,67 / (0,50 + 0,67) = 0,57. Rata-rata yang menyeimbangkan recall dan precision.

ROC-AUC. Peluang model memberi prob_bad lebih tinggi kepada pemohon bad dibanding pemohon good, kalau keduanya diambil acak. Nilai 0,5 sama dengan menebak, dan 1,0 berarti sempurna. Contoh: dua pemohon bad mendapat skor 0,8 dan 0,4, dan tiga pemohon good mendapat 0,6, 0,3, dan 0,1. Dari 6 pasangan bad–good, 5 pasangan menempatkan bad lebih tinggi, jadi AUC = 5 / 6 = 0,83. AUC tidak bergantung pada threshold.

MLOps

MLflow. Buku catatan eksperimen yang otomatis. Setiap kali kamu melatih satu model, MLflow mencatat setelan, hasil, dan file modelnya. Semua percobaan bisa dibandingkan berdampingan di browser.

Istilah	Artinya	Contoh di tugas ini
Experiment	Folder yang mengelompokkan percobaan	`credit-risk`
Run	Satu kali percobaan	run `random_forest`
Param	Setelan yang dipilih sebelum melatih	`max_depth=6`, `data_version=…`

Istilah	Artinya	Contoh di tugas ini
Metric	Angka hasil setelah melatih	`roc_auc`, `recall_bad`
Artifact	File yang disimpan bersama run	model sklearn beserta signature-nya
Tag	Label bebas	`reason=commit baru`, `quality_gate=lulus`

Perbandingan yang adil (controlled experiment). Ketiga run memakai data, split, dan preprocessing yang sama; hanya jenis modelnya yang berbeda. Kalau model dan data diubah bersamaan, kamu tidak tahu mana yang membuat hasil berubah.

Versi data dan hash. Hash adalah "sidik jari" isi file: deretan karakter pendek yang berubah total walaupun hanya satu angka di file yang berubah. Hash dicatat sebagai param `data_version` di setiap run, jadi kamu selalu tahu model dilatih dengan data versi yang mana.

Quality gate dan exit code. Quality gate adalah "satpam" yang memeriksa angka evaluasi sebelum model boleh dipakai. Di tugas ini batasnya `roc_auc ≥ 0,70` **dan** `recall_bad ≥ 0,50`. Hasilnya dilaporkan lewat exit code: 0 berarti lulus, 1 berarti gagal. Pipeline CI/CD membaca exit code ini untuk memutuskan lanjut atau berhenti.

roc_auc	recall_bad	Hasil gate
0,76	0,72	Lulus (exit 0)
0,77	0,36	Gagal (exit 1): AUC tinggi, tetapi terlalu banyak pemohon bad yang lolos

Model registry, versi, dan alias. Registry adalah lemari arsip model. Setiap model yang didaftarkan ke nama `credit-risk` mendapat nomor versi (1, 2, 3, …). Alias `champion` adalah label yang menunjuk versi yang sedang dipakai di produksi. Kode serving selalu memuat `models:/credit-risk@champion`, jadi mengganti model cukup dengan memindahkan alias tanpa mengubah kode.

Promote dan rollback. Promote berarti memindahkan alias `champion` ke versi baru. Di tugas ini promote hanya terjadi kalau model baru lulus gate **dan** AUC-nya lebih tinggi dari champion lama. Rollback adalah kebalikannya: alias dikembalikan ke versi sebelumnya.

Pipeline commit-to-deploy (CI/CD lokal). Urutan otomatis yang dijalankan setiap ada perubahan: test → train → quality gate → pilih → register → promote. Kalau satu langkah gagal, langkah berikutnya tidak dijalankan. Di industri urutan ini biasanya dijalankan oleh GitHub Actions atau Jenkins. Di tugas ini cukup dijalankan dengan `python src/pipeline.py`.

pytest. Test otomatis, yaitu potongan kode yang memastikan program berperilaku sesuai harapan. Contohnya, quality gate harus menolak model dengan AUC 0,60:

```
def test_quality_gate_bisa_gagal():
    assert check({"roc_auc": 0.60, "recall_bad": 0.60})[0] is False
```

Bagian B — MLOps model credit risk (Pertemuan 27)

Targetnya: satu perintah `python src/pipeline.py` menjalankan test, melatih 3 model, mencatat semuanya di MLflow, menolak model yang tidak lulus quality gate, lalu mempromosikan model terbaik yang lulus menjadi `credit-risk@champion`.

1. **Data** (`data.py`). `load_data` membaca dan merapikan CSV sesuai docstring, `split` membagi data 70/30 secara stratified, dan `data_version` menghitung SHA-256 isi file (12 karakter pertama). `validate_record` baru dipakai di Bagian C, tetapi ditulis di sini.
2. **Model dan metrik** (`train.py`). `build_model` menyusun Pipeline preprocessing + model untuk setiap kandidat di `CANDIDATES` (`logreg` , `random_forest` , `gradient_boosting` ; hyperparameter-nya sudah ditentukan). `evaluate` menghitung `roc_auc` , `recall_bad` , `precision_bad` , dan `f1_bad` .
3. **Quality gate** (`quality_gate.py` → `check` , `main`). Bandingkan setiap metrik dengan `GATES` , lalu kembalikan `(lulus, daftar_alasan)` . Saat dijalankan sendiri, file ini keluar dengan exit 0 (lulus) atau 1 (gagal).
4. **Pipeline** (`pipeline.py`). Tulis lima langkahnya sesuai docstring:
   - `step_test` menjalankan pytest.
   - `step_train` melatih ketiga kandidat; setiap kandidat satu MLflow run lengkap dengan params, metrics, tag, dan model.
   - `step_gate` memeriksa setiap kandidat dengan quality gate.
   - `step_select` memilih AUC tertinggi **di antara yang lulus**.
   - `step_register_promote` mendaftarkan model ke registry dan memindahkan alias `champion` hanya kalau model baru lebih baik.
5. **Jalankan dan buktikan.**
   - `python src/pipeline.py --reason "commit baru"` , lalu `python src/quality_gate.py` (harus keluar dengan exit 0).
   - Buka `mlflow ui --backend-store-uri sqlite:///mlflow.db` . Bandingkan ketiga run, lalu simpan screenshot-nya ke `screenshots/mlflow_runs.png` . Screenshot halaman model `credit-risk` yang menunjukkan alias champion juga, simpan ke `screenshots/mlflow_registry.png` .

- Buktikan gate bisa gagal: ubah sementara `GATES` menjadi `roc_auc: 0.95`, jalankan `python src/pipeline.py --skip-tests` (test-nya memang akan gagal dengan batas ini), lalu tempel output-nya di README. Setelah itu kembalikan ke nilai semula.
- Jalankan pipeline sekali lagi tanpa mengubah apa pun, lalu jelaskan di README kenapa versi baru tidak dipromosikan.

Saat mengerjakan, perhatikan model dengan AUC tertinggi. Apakah ia lulus gate? Jelaskan di README apa artinya bagi bank kalau model itu yang dipakai.

Istilah Bagian C (monitoring dan drift)

Serving dan instrumentasi. Serving adalah menjalankan model untuk melayani permintaan nyata, satu pemohon per request. Instrumentasi berarti menambahkan pencatatan ke setiap request. Contoh satu baris `logs/predictions.csv` (kolom fitur dipotong):

```
timestamp,request_id,batch,status,error,latency_ms,prob_bad,decision,age,cred
it_amount,...
2026-11-02T09:15:00.120,1-0007,minggu_1,ok,,19.4,0.2381,setuju,35,2100,...
2026-11-09T10:02:11.551,2-0010,minggu_2,error,field kosong:
['age'],0.1,,,,4300,...
```

Batch dan referensi. Batch adalah sekelompok request dalam satu periode, misalnya `minggu_1` dan `minggu_2`. Referensi adalah data pembanding yang dianggap "normal", yaitu data training model. Setiap batch dibandingkan dengan referensi.

Latency p50 dan p95. p50 (median) adalah waktu proses yang dilewati separuh request. p95 adalah batas waktu yang dilampaui hanya oleh 5% request terlambat. Contoh: dari 100 request, 95 selesai dalam kira-kira 20 ms dan 5 butuh 300 ms. Rata-ratanya sekitar 34 ms, padahal tidak ada satu pun request yang selesai di kisaran itu. Karena rata-rata bisa menyesatkan, alert memakai p95.

Throughput. Jumlah request yang dilayani per detik (requests per second). Contoh: 300 request selesai dalam 6 detik, jadi throughput-nya 50 req/s.

Error rate. Persentase request yang gagal diproses, misalnya karena umur kosong atau jumlah pinjaman negatif. Contoh: 6 error dari 300 request = 2%.

Quality metric dan label yang datang belakangan. Saat kredit diajukan, kita belum tahu apakah pemohon akan gagal bayar. Labelnya baru diketahui berbulan-bulan kemudian. Di simulasi ini label disimpan terpisah di `logs/labels.csv`, lalu digabungkan untuk menghitung AUC per batch, seolah-olah labelnya sudah datang.

Data drift dan prediction drift. Data drift terjadi saat input berubah dibanding referensi, misalnya pemohon minggu ini lebih muda dan meminjam lebih besar. Prediction drift terjadi saat output model berubah, misalnya porsi keputusan "tolak" naik dari 27% ke 42%. Drift belum tentu berarti model rusak; bisa jadi populasinya yang berubah. Karena itu drift harus diperiksa bersama quality metric.

KS test dan p-value (fitur angka). KS test membandingkan dua kumpulan angka untuk menjawab: apakah keduanya berasal dari sebaran yang sama? Hasilnya p-value. Kalau p-value < 0,05, perbedaannya kemungkinan besar nyata, bukan kebetulan. Contoh: umur referensi kebanyakan 25–45 tahun, sedangkan umur minggu_2 kebanyakan 19–30 tahun. p-value-nya akan sangat kecil.

PSI (Population Stability Index, fitur angka). Angka yang mengukur seberapa besar sebaran bergeser. Caranya: bagi nilai ke beberapa bin (rentang), lalu bandingkan proporsi data di setiap bin. Template memakai 10 bin dari kuantil referensi. Patokan umumnya: di bawah 0,1 stabil, 0,1–0,2 mulai bergeser, dan di atas 0,2 bergeser besar.

```
PSI = jumlah untuk setiap bin i dari  (q_i − p_i) × ln(q_i / p_i)
```

Di sini p adalah proporsi referensi dan q proporsi batch. Contoh dengan 5 bin umur dan 10 pemohon per data:

Bin umur	Referensi (p)	Batch (q)	(q − p) × ln(q/p)
19–25	1 → 0,1	2 → 0,2	0,069
26–30	1 → 0,1	3 → 0,3	0,220
31–40	2 → 0,2	2 → 0,2	0,000
41–50	3 → 0,3	2 → 0,2	0,041
51+	3 → 0,3	1 → 0,1	0,220
Total PSI			**0,549**

Nilai 0,549 lebih besar dari 0,2, jadi drift. Angka kecil 0,0001 (`EPS`) ditambahkan ke setiap proporsi karena bin yang kosong membuat ln(0) tidak bisa dihitung.

Jensen-Shannon distance (fitur kategori). Ukuran beda dua distribusi kategori, dari 0 (sama persis) sampai 1 (beda total). Batas alert di tugas ini 0,1. Contoh untuk `purpose`:

purpose	Referensi	Batch A	Batch B
car	50%	45%	30%

purpose	Referensi	Batch A	Batch B
radio/TV	30%	33%	20%
business	20%	22%	50%
JS distance		**0,04** (aman)	**0,27** (drift)

Alert. Peringatan otomatis saat angka melewati batas. Alert tidak memperbaiki apa pun; ia hanya memberi tahu bahwa ada yang perlu dicek.

Dashboard. Satu gambar yang merangkum kondisi model: distribusi fitur, PSI, Jensen-Shannon, error rate, dan proporsi keputusan per batch. Di tugas ini dashboard berupa `reports/monitoring_dashboard.png`.

Remediation, retraining trigger, dan SOP. Remediation adalah tindakan perbaikan setelah alert, misalnya memperbaiki validasi input, retrain, atau rollback. Retraining trigger adalah penanda otomatis bahwa retrain perlu dipertimbangkan; di tugas ini berupa `reports/retrain_trigger.json`. SOP (Standard Operating Procedure) adalah langkah baku yang diikuti saat alert menyala, supaya siapa pun yang bertugas memeriksa hal yang sama dengan urutan yang sama.

Bagian C — Monitoring dan drift (Pertemuan 28)

Targetnya: monitoring tidak membunyikan alert untuk `minggu_1` yang normal, tetapi menangkap masalah di `minggu_2`. Kamu juga harus bisa menjelaskan apa yang terjadi dan apa yang harus dilakukan.

Simulasinya mengirim 300 request `minggu_1` (data test apa adanya) dan 300 request `minggu_2` ke model champion. Data `minggu_2` sengaja digeser: pemohon 10 tahun lebih muda, pinjaman 1,8 kali lebih besar, tenor 12 bulan lebih lama, 35% tujuannya diubah menjadi `business`, dan 6 input dibuat rusak. Aturan persisnya ada di docstring `simulate.py`; ikuti apa adanya supaya hasilmu bisa dibandingkan dengan teman.

1. **Serving** (`serve.py`). `decide` menerapkan kebijakan K05. `CreditScorer` memuat `models:/credit-risk@champion`, memvalidasi setiap input, menghitung `prob_bad` dan keputusan, mengukur latency, lalu menulis satu baris log per request. Input yang rusak tidak boleh membuat program crash; catat sebagai `status=error` beserta pesannya.
2. **Simulasi** (`simulate.py` → `make_week2`, `main`). Buat data `minggu_2`, sisipkan 6 input rusak, kirim kedua minggu lewat `CreditScorer`, lalu simpan label ke `logs/labels.csv`.

3. **Drift** (`monitor.py` → `psi` , `js_distance` , `drift_table`). PSI dan KS test untuk fitur angka, Jensen-Shannon untuk fitur kategori, dengan referensi berupa data training.
4. **Metrik operasional dan kualitas** (`ops_metrics`). Untuk setiap batch, hitung jumlah request, error rate, p50 dan p95 latency, throughput, rata-rata `prob_bad` , porsi keputusan "tolak", dan AUC dari label.
5. **Aturan alert** (`alerts`). Buat pesan ALERT kalau error rate > 1%, p95 latency > 200 ms, AUC < 0,70, atau ada fitur yang drift (PSI > 0,2, p-value KS < 0,05, atau Jensen-Shannon > 0,1).
6. **Dashboard dan laporan** (`dashboard` , `main`). Jalankan `python src/simulate.py` , lalu `python src/monitor.py` . Hasilnya `reports/monitoring_report.md` , `reports/monitoring_dashboard.png` , `reports/drift_metrics.csv` , dan `reports/retrain_trigger.json` .
7. **Tulis SOP.md** (template sudah ada, maksimal 1 halaman). Isinya: pemicu, pemeriksaan pertama, cara membedakan input rusak, populasi berubah, dan model memburuk, tindakan untuk masing-masing (termasuk kapan retrain dan kapan rollback), langkah retrain beserta verifikasinya, dan pencatatan.

Di README, jawab tiga pertanyaan ini dengan angka dari laporanmu:

- Fitur mana yang drift di `minggu_2` , dan seberapa besar?
- Apakah AUC `minggu_2` turun di bawah 0,70? Kalau tidak, apakah drift-nya tetap perlu ditindaklanjuti? Kenapa?
- Apakah kamu akan retrain sekarang? Data apa yang dibutuhkan supaya retrain-nya berarti?

Struktur repo dan pengumpulan

Kumpulkan satu repository GitHub bernama `tugas-26-28-<nama>` . Strukturnya mengikuti template:

```
tugas-26-28-<nama>/
├── README.md                  # diisi: hasil, keputusan, limitation
├── SOP.md                     # diisi: maksimal 1 halaman
├── requirements.txt
├── .gitignore                 # .venv/, mlruns/, mlflow.db, .env
├── data/
│   ├── rag/docs/K01.md … K08.md
│   ├── rag/eval.csv
│   └── credit/german_credit_data.csv
├── src/                       # rag, rag_eval, data, train, quality_gate,
pipeline, serve, simulate, monitor
├── tests/                     # test_rag.py, test_credit.py
├── logs/                      # predictions.csv, labels.csv
├── reports/                   # rag_eval_*.csv, candidates.json,
best_metrics.json,
│                              # monitoring_report.md,
monitoring_dashboard.png, drift_metrics.csv, retrain_trigger.json
└── screenshots/               # mlflow_runs.png, mlflow_registry.png
```

README wajib berisi:

- [ ] Tabel hasil keempat mode RAG, ambang fallback yang dipakai, dan mode yang dipilih beserta alasannya
- [ ] Tabel ketiga model (metrik dan hasil gate), `data_version`, champion beserta alasannya, dan bukti gate bisa gagal
- [ ] Dashboard monitoring, alert yang menyala, dan jawaban tiga pertanyaan di Bagian C
- [ ] Minimal 2 limitation dan 1 eksperimen berikutnya

Pengumpulan: kirim link repository ke mentor. Kalau repository-nya private, beri akses ke mentor.

Deadline: 1 minggu sejak tugas dibagikan

Jangan meng-commit `.venv/`, `mlruns/`, `mlflow.db`, file `.env`, atau API key apa pun. File `.gitignore` di template sudah mengaturnya.

Penilaian

Nilai lulus adalah 70 dari 100.

Aspek	Bobot	Yang diperiksa
Functionality	25%	Semua perintah berjalan dari clone bersih, 8 test lulus, model champion terdaftar, laporan monitoring terbentuk
Correctness	20%	RRF, Hit@3, metrik model, PSI, Jensen-Shannon, dan aturan alert dihitung dengan benar; sitasi menunjuk dokumen dan pasal yang tepat
Engineering Quality	20%	Struktur repo rapi, tidak ada secret atau file besar yang ter-commit, kode tetap terbaca
Evaluation & Reliability	20%	Perbandingan yang adil (4 mode RAG, 3 model), quality gate terbukti bisa gagal, failure test (input kosong dan input rusak), alert minggu_1 bersih
Documentation	15%	README menjawab semua pertanyaan dengan angka; SOP bisa langsung dipakai orang lain

Bonus (maksimal +10, pilih yang kamu minati):

- Jawaban RAG dibuat oleh LLM dari chunk yang ditemukan, tetap dengan sitasi dan tanpa mengarang di luar konteks
- Retrain dengan menambahkan data `minggu_2` yang sudah berlabel, lalu bandingkan AUC-nya dengan champion lama
- Serving dengan FastAPI, atau dashboard Prometheus + Grafana, atau laporan drift dengan Evidently
- Versi data dengan DVC sebagai pengganti hash manual

Tips:

- Kerjakan berurutan: A, lalu B, lalu C. Jalankan `python -m pytest -q` setiap kali satu file selesai.
- Muat model embedding dan reranker sekali saja di `_index()`. Jangan memuatnya ulang di dalam `answer()`, karena latency bisa melonjak menjadi beberapa detik per pertanyaan.
- Kalau PSI keluar `inf` atau `nan`, cek apakah `EPS` sudah ditambahkan.
- Kalau `mlflow ui` kosong, pastikan kamu menjalankannya dari folder repo dengan `--backend-store-uri sqlite:///mlflow.db`.
- Hasil yang di bawah target tetap dinilai, asalkan dijelaskan jujur beserta penyebabnya.

Lampiran — Data Bagian A

Semua dokumen ini fiktif dan sudah tersedia di template. Lampiran ini disertakan supaya kamu bisa membaca isinya tanpa membuka file.

Dokumen kebijakan kredit (data/rag/docs)

```
# K01 — Syarat Pemohon Kredit
Pasal 1: Pemohon kredit tanpa agunan (KTA) harus berusia 21 sampai 60 tahun
pada saat pengajuan.
Pasal 2: Pemohon wajib memiliki KTP yang masih berlaku dan nomor telepon
aktif atas nama sendiri.
Pasal 3: Pemohon karyawan harus memiliki penghasilan tetap minimal 6 bulan di
tempat kerja saat ini.

# K02 — Plafon dan Tenor
Pasal 1: Plafon KTA paling tinggi Rp200.000.000 per nasabah.
Pasal 2: Tenor kredit dapat dipilih antara 12 sampai 60 bulan.
Pasal 3: Total cicilan seluruh pinjaman nasabah tidak boleh melebihi 30% dari
penghasilan bulanan (debt burden ratio).

# K03 — Suku Bunga
Pasal 1: Suku bunga efektif KTA berkisar 11% sampai 16% per tahun, ditentukan
oleh skor risiko pemohon.
Pasal 2: Nasabah yang menerima gaji melalui rekening Bank Nusantara (payroll)
mendapat potongan bunga 1% per tahun.
Pasal 3: Suku bunga bersifat tetap selama tenor dan tidak berubah walaupun
suku bunga acuan naik.

# K04 — Dokumen Pengajuan
Pasal 1: Dokumen wajib terdiri dari KTP, slip gaji 3 bulan terakhir, dan
rekening koran 3 bulan terakhir.
Pasal 2: NPWP wajib dilampirkan untuk pengajuan dengan plafon di atas
Rp50.000.000.
Pasal 3: Pemohon wiraswasta mengganti slip gaji dengan laporan keuangan usaha
6 bulan terakhir.

# K05 — Skor Risiko dan Keputusan Kredit
Pasal 1: Model skor risiko menghitung probabilitas gagal bayar pemohon dari
data aplikasi.
Pasal 2: Probabilitas di bawah 0,3 disetujui otomatis, 0,3 sampai 0,6
ditinjau analis kredit, dan di atas 0,6 ditolak.
Pasal 3: Model skor risiko dievaluasi ulang setiap 6 bulan atau segera
setelah tim data mendeteksi drift.

# K06 — Keterlambatan dan Denda
```

```
Pasal 1: Keterlambatan pembayaran cicilan dikenakan denda 0,1% per hari dari
cicilan yang tertunggak.
Pasal 2: Kredit yang menunggak lebih dari 90 hari digolongkan sebagai kredit
macet.
Pasal 3: Nasabah yang mengalami kesulitan bayar dapat mengajukan
restrukturisasi kredit melalui cabang.

# K07 — Pelunasan Dipercepat
Pasal 1: Pelunasan dipercepat dapat dilakukan setelah nasabah membayar
minimal 6 kali cicilan.
Pasal 2: Pelunasan dipercepat dikenakan penalti 2% dari sisa pokok pinjaman.
Pasal 3: Pelunasan sebagian minimal sebesar 3 kali cicilan bulanan.

# K08 — Perlindungan Data dan Pengaduan
Pasal 1: Data pemohon hanya dipakai untuk penilaian kredit dan tidak
dibagikan ke pihak ketiga tanpa persetujuan tertulis.
Pasal 2: Pengaduan nasabah dijawab paling lambat 10 hari kerja sejak
diterima.
Pasal 3: Pemohon yang ditolak berhak meminta penjelasan tertulis mengenai
alasan penolakan.
```

Pertanyaan uji (data/rag/eval.csv)

```
question,expected_doc
Berapa umur minimal untuk mengajukan KTA?,K01
Berapa lama harus bekerja sebelum boleh mengajukan kredit?,K01
Berapa pinjaman maksimal yang bisa diajukan?,K02
Berapa batas cicilan dibanding gaji?,K02
Apakah bunga bisa naik di tengah masa pinjaman?,K03
Apakah karyawan payroll dapat potongan bunga?,K03
Kapan NPWP wajib dilampirkan?,K04
"Saya wiraswasta tanpa slip gaji, dokumen apa penggantinya?",K04
Skor berapa yang langsung ditolak?,K05
Berapa denda kalau telat bayar cicilan?,K06
Berapa penalti kalau melunasi pinjaman lebih cepat?,K07
Berapa lama pengaduan saya akan dijawab?,K08
Siapa pemenang Piala Dunia 2022?,NONE
Bagaimana cara membuat nasi goreng?,NONE
Berapa harga emas hari ini?,NONE
```

Dataset Bagian B dan C: [German Credit Data with Risk] (Kaggle), versi ringkas dari dataset Statlog German Credit milik UCI.